Nimby - Inteligencja obliczeniowa w analizie danych cyfrowych

Paweł Adamczyk Dominika Bujnarowska

Opis projektu: Niniejszy raport przedstawia analizę wpływu losowości na skuteczność algorytmu przeszukującego Negamax w kontekście gier turowych. W ramach eksperymentów zaimplementowano klasyczną grę Nim oraz jej probabilistyczny wariant (Nimby), a następnie zbadano skuteczność algorytmów decyzyjnych w zależności od głębokości przeszukiwania drzewa gry.

1 Opis Gry i Środowiska Eksperymentalnego

Przedmiotem zrealizowanych eksperymentów jest klasyczna gra matematyczna Nim oraz wspomniana probabilistyczna modyfikacja o nazwie „Nimby". Stan początkowy gry definiują trzy stosy, zawierające domyślnie odpowiednio 3, 4 i 5 elementów. Rozgrywka toczy się w systemie turowym, w którym dwaj gracze na przemian usuwają ze stołu dowolną, niezerową liczbę elementów z wybranego przez siebie, pojedynczego stosu. Zgodnie z przyjętą konwencją, zwycięzcą zostaje gracz, który zdejmie z planszy ostatni element (w środowisku programistycznym zrealizowano to poprzez przypisanie wartości punktowej -100 graczowi, który w swojej turze nie ma już do dyspozycji żadnego legalnego ruchu). Warto zaznaczyć, że specyfika i zasady gry Nim wykluczają możliwość zakończenia partii remisem.

Na potrzeby projektu zaimplementowano wariant probabilistyczny – Nimby. Różnica względem klasycznej wersji polega na wprowadzeniu czynnika losowego: z prawdopodobieństwem wynoszącym 10% gracz wykonujący ruch zostaje zmuszony do wzięcia z wybranego stosu o jeden element mniej, niż pierwotnie zadeklarował (sytuacja ta ma miejsce wyłącznie, gdy gracz planował zabrać więcej niż jeden element). Wprowadza to do rozgrywki szum informacyjny, z którym musi poradzić sobie sztuczna inteligencja.

Do symulacji wykorzystano framework `easyAI` oraz wbudowany weń algorytm przeszukiwania drzewa gry – Negamax.

2 Metodologia Eksperymentów

Celem przeprowadzonych badań było zestawienie skuteczności agentów AI operujących na różnej głębokości przeszukiwania drzewa decyzyjnego (parametr *Depth* równy 3, 4 oraz 5).

Turnieje przeprowadzono według następującego schematu:

- Każde starcie (np. Depth 3 vs Depth 5) składało się z równej liczby 100 partii.

- Aby wyeliminować przewagę wynikającą z faktu bycia graczem rozpoczynającym, po rozegraniu 50 gier następowała zamiana kolejności startowej.
- Zliczono wyłącznie wygrane i przegrane (remisy, jak wspomniano wcześniej, nie występują w tej grze).
- Przeprowadzono dwa równoległe zestawy testów: dla środowiska w pełni deterministycznego (Nim) oraz probabilistycznego (Nimby 10%).

3 Napotkane Problemy Badawcze i Ich Rozwiązania

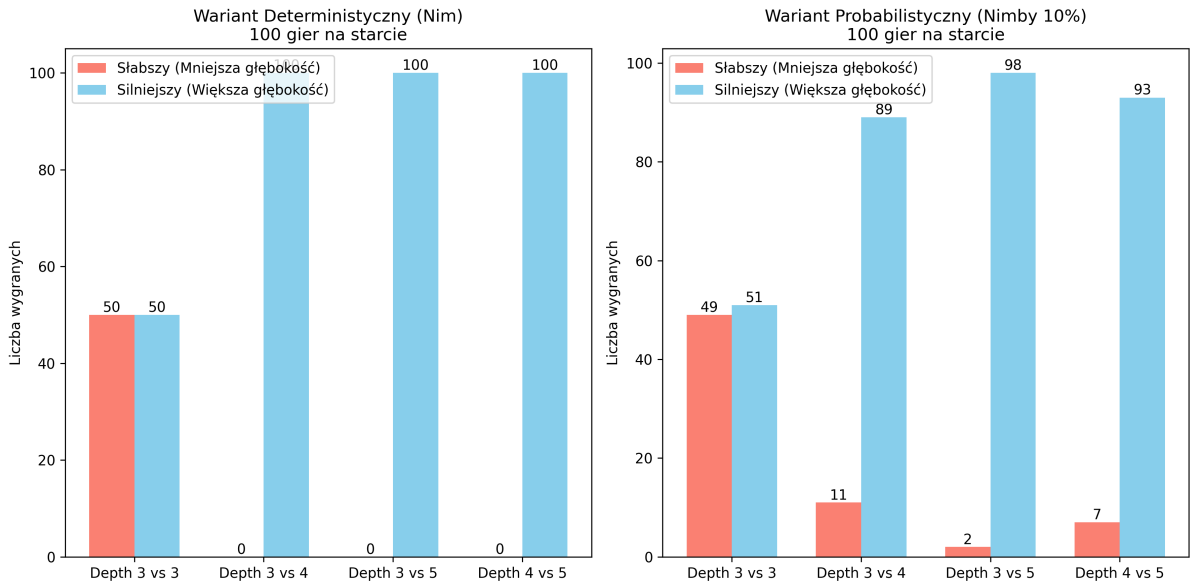
Podczas implementacji wariantu probabilistycznego natrafiono na krytyczny problem architektoniczny wynikający z konstrukcji frameworka `easyAI`. Algorytm Negamax, stosowany do wyznaczania optymalnego ruchu, opiera się na symulowaniu przyszłych stanów gry za pomocą metod `make_move` (wykonanie ruchu) oraz `unmake_move` (cofnięcie ruchu). Algorytm ten zakłada, że środowisko jest w pełni deterministyczne – wywołanie akcji i jej cofnięcie musi precyzyjnie przywrócić stan planszy do punktu wyjścia.

Pierwotna próba implementacji polegała na umieszczeniu 10% szansy na zmianę decyzji bezpośrednio wewnątrz metody `make_move`. Skutkowało to desynchronizacją drzewa stanów podczas działania sztucznej inteligencji. Gdy algorytm symulował wzięcie np. 3 elementów, losowość mogła sprawić, że odjęto jedynie 2. Jednakże podczas wycofywania ruchu (`unmake_move`), algorytm dodawał z powrotem 3 elementów. Wskutek tysięcy takich operacji wykonywanych w ułamku sekundy podczas analizy drzewa, wirtualne stosy zaczęły przyjmować wartości abstrakcyjne (rosły w nieskończoność lub spadały poniżej zera).

Rozwiązanie: Problem zażegnano poprzez ścisłą separację procesu decyzyjnego AI od logiki silnika gry. Wewnątrz klasy definiującej zasady zachowano rygorystyczny determinizm, pozwalając algorytmowi Negamax na prawidłowe symulacje. Zjawisko losowości przeniesiono do zewnętrznej pętli turniejowej. Ewaluacja 10% prawdopodobieństwa błędu jest aplikowana *in-flight* (w locie), tuż po wyznaczeniu optymalnego ruchu przez algorytm, ale przed jego ostatecznym zatwierdzeniem i zaaplikowaniem na planszy.

4 Wyniki Eksperymentów

Poniższy wykres prezentuje zestawienie liczby zwycięstw odniesionych przez poszczególnych agentów w serii 100 gier.



Rysunek 1: Porównanie wyników starć agentów Negamax na wariancie deterministycznym i probabilistycznym.

4.1 Analiza Wariantu Deterministycznego (Zwykły Nim)

Wyniki uzyskane w klasycznym wariancie gry perfekcyjnie ilustrują bezwzględną dominację agentów o wyższej głębokości przeszukiwania (tzw. *Depth*):

- **Depth 3 vs 3 (50 do 50):** Starcie identycznych algorytmów dało idealnie zrównoważony wynik. O wygranej w pojedynczej partii decydowało wyłącznie to, kto rozpoczynał grę (zamiana nastąpiła w połowie turnieju).
- **Depth 3 vs 4 (0 do 100):** Agent o widoczności 4 ruchów wygrał każdą ze 100 partii.
- **Depth 3 vs 5 (0 do 100) oraz Depth 4 vs 5 (0 do 100):** Całkowita dominacja silniejszego algorytmu. Brak czynnika losowego sprawia, że jeśli silniejszy algorytm potrafi zidentyfikować strategię wygrywającą, słabszy przeciwnik nie ma matematycznej możliwości na zmianę biegu wydarzeń.

4.2 Analiza Wariantu Probabilistycznego (Nimby 10%)

Wprowadzenie szumu w postaci zaledwie 10-procentowej szansy na błąd gracza (zmniejszenie pobieranej puli o 1) drastycznie zmieniło morfologię wyników:

- **Depth 3 vs 3 (49 do 51):** Czynnik losowy nieznacznie zaburzył idealny remis z wersji deterministycznej, faworyzując minimalnie jednego z graczy z powodu asymetrycznego wpływu pecha w kluczowych momentach partii.
- **Depth 3 vs 4 (11 do 89):** Agent widzący płycej zdołał urwać przeciwnikowi 11 zwycięstw.
- **Depth 3 vs 5 (2 do 98):** Przewaga Depth 5 jest tak potężna, że nawet losowość rzadko potrafi wytrącić go z równowagi, niemniej jednak Depth 3 zdołał wygrać 2 gry.

- **Depth 4 vs 5 (7 do 93):** Agent z Depth 4 odebrał 7 zwycięstw silniejszemu rywalowi.

5 Wnioski

Z przeprowadzonych eksperymentów płyną dwa główne wnioski dotyczące działania algorytmów przeszukujących w grach logicznych. Po pierwsze, w grach o pełnej informacji i zerowej losowości (jak klasyczny Nim), przewaga wynikająca z głębszej analizy drzewa decyzyjnego jest absolutna. Słabszy algorytm popełnia błędy, które silniejszy bezlitośnie i bezbłędnie wykorzystuje, nie dając szans na zmianę wyniku (rezultaty 100 do 0).

Po drugie, wprowadzenie do gry probabilistyki stanowi koło ratunkowe dla słabszych systemów. Algorytm Negamax z natury zakłada idealne wykonanie zaplanowanej akcji. Kiedy losowość psuje precyzyjny plan (zmuszając silniejszego agenta do wzięcia innej liczby elementów), stan planszy zmienia się na taki, którego faworyt nie przewidział jako optymalnego. Te losowe potknięcia tworzą "okna możliwości", które słabszy agent sporadycznie potrafi wykorzystać, odwracając losy partii. Badania wykazały, że nawet niewielkie (10%) prawdopodobieństwo błędu skutecznie rozbija deterministyczny monopol na wygraną.

6 Optymalizacja Algorytmu: Odcięcie Alfa-Beta i Złożoność Czasowa

Kolejnym etapem badań była weryfikacja wpływu optymalizacji algorytmu za pomocą cięć Alfa-Beta na skuteczność decyzyjną oraz czas wykonywania obliczeń. Standardowy algorytm Negamax został zestawiony z jego surową wersją (pozbawioną mechanizmu Alfa-Beta). Eksperymenty przeprowadzono dla dwóch wariantów głębokości (Depth 4 i Depth 5), rozgrywając po 20 partii dla każdego zestawienia (ze zmianą gracza rozpoczynającego po 10 grach). Wbudowany w skrypt moduł zliczał łączny czas procesora przeznaczony na ewaluację drzewa gry dla każdego z agentów.

6.1 Zestawienie Wyników

Tabela 1 prezentuje zebrane dane z podziałem na środowisko deterministyczne i probabilistyczne. Czas wyraża łączny czas myślenia danego algorytmu w trakcie trwania całego starcia (20 gier).

Tabela 1: Porównanie wygranych i czasów obliczeń algorytmów z/bez odcięcia Alfa-Beta (20 gier na starcie)

Środowisko	**Starcie (Algorytm A vs Algorytm B)**	**Wygrane A**	**Wygrane B**	**Czas A [s]**	**Czas B [s]**
Deterministyczne	Negamax(AB, 4) vs Negamax(NoAB, 4)	10	10	0.050	0.238
	Negamax(AB, 5) vs Negamax(NoAB, 5)	10	10	0.127	0.925
Probabilistyczne	Negamax(AB, 4) vs Negamax(NoAB, 4)	10	10	0.097	0.408
	Negamax(AB, 5) vs Negamax(NoAB, 5)	10	10	0.145	0.953
	Negamax(AB, 4) vs Negamax(AB, 5)	1	19	0.042	0.129

6.2 Analiza i Wnioski z Pomiarów Czasu

Na podstawie danych z Tabeli 1 można sformułować dwa fundamentalne wnioski dotyczące działania optymalizacji Alfa-Beta:

- **Bezbłędność logiczna i identyczność decyzji:** We wszystkich starciach pomiędzy agentem z optymalizacją Alfa-Beta (AB), a agentem sprawdzającym całe drzewo (NoAB), zachodzi idealny remis w wygranych (10 do 10). Stanowi to empiryczny dowód na to, że odcięcie Alfa-Beta nie zmienia ostatecznej decyzji minimaxowej. Algorytm pomija wyłącznie te gałęzie drzewa decyzyjnego, które matematycznie zostały już zdyskwalifikowane przez wcześniej zbadane warianty.

- **Wydajność i złożoność czasowa:** Optymalizacja Alfa-Beta radykalnie zmniejsza czas potrzebny na wybranie ruchu. Dla głębokości przeszukiwania równej 4, algorytm bez odcięcia potrzebował niemal pięciokrotnie więcej czasu (0.238 s w stosunku do 0.050 s). Wraz ze wzrostem głębokości przeszukiwania do 5, różnica ta urosła drastycznie – wariant NoAB liczył ruchy prawie ośmiokrotnie dłużej (0.925 s wobec 0.127 s). Potwierdza to eksponencjalną redukcję odwiedzanych węzłów, jaką zapewnia ta technika. Warto również zauważyć, że obliczenia w wariancie probabilistycznym trwały delikatnie dłużej, co wynika z konieczności przetworzenia "w locie" zepsutych, podmienionych przez losowość ruchów.

Dodatkowe starcie dwóch różnych głębokości (Depth 4 vs Depth 5 z włączonym Alfa-Beta) w środowisku probabilistycznym potwierdziło wcześniejsze obserwacje z pierwszej fazy eksperymentów – znacznie silniejszy agent z Depth 5 zdominował przeciwnika (19 wygranych do 1), zużywając przy tym proporcjonalnie więcej czasu na obliczenia wynikające z przeglądu głębszych warstw drzewa (0.129 s kontra 0.042 s).

7 Algorytm Expectiminimax z Odcięciem Alfa-Beta

Ostatnim, najbardziej zaawansowanym etapem projektu była implementacja algorytmu Expectiminimax z uwzględnieniem odcięcia Alfa-Beta i porównanie jego wydajności z klasycznym algorytmem Negamax[cite: 21]. Zwykły algorytm minimaxowy (w tym wariant Negamax) zakłada w pełni deterministyczne środowisko – ewaluuje planszę przyjmując, że zaplanowany ruch powiedzie się w 100%. W środowisku probabilistycznym (Nimby) takie podejście może prowadzić do wyboru strategii "kruchych", które załamują się przy wystąpieniu pechowego zdarzenia.

Algorytm Expectiminimax rozwiązuje ten problem poprzez wprowadzenie tzw. węzłów szansy (ang. *chance nodes*) . Wartość takiego węzła nie jest prostym maksimum z dostępnych opcji, lecz wartością oczekiwaną obliczaną na podstawie prawdopodobieństwa wystąpienia danego stanu gry. Dla wariantu Nimby z 10% szansą na wzięcie o jeden element mniej, wartość oczekiwana węzła obliczana jest ze wzoru:

$$E = 0.9 \cdot V_{normal} + 0.1 \cdot V_{slip}$$

gdzie V_{normal} to ewaluacja pełnego ruchu, a V_{slip} to ewaluacja ruchu pomniejszonego o jeden element.

7.1 Zestawienie Wyników Wydajnościowych

W celu weryfikacji algorytmu rozegrano turnieje (po 100 partii każdy) w wariancie probabilistycznym Nimby, zestawiając standardowy algorytm Negamax z nową implementacją Expectiminimax dla głębokości (Depth) 4 oraz 5. Wyniki przedstawiono w Tabeli 2.

Tabela 2: Porównanie skuteczności i czasu obliczeń: Negamax vs Expectiminimax (100 gier na starcie, wariant Nimby 10%)

Starcie (100 partii)	Wygrane Negamax	Wygrane Expectiminimax	Czas Negamax [s]	Czas Expectiminimax [s]
Depth 4	44	56	0.237	0.599
Depth 5	48	52	0.661	2.368

7.2 Analiza i Wnioski

Analiza zebranych danych pozwala na sformułowanie następujących wniosków końcowych:

- **Przewaga strategiczna (świadomość ryzyka):** Algorytm Expectiminimax w obu testowanych scenariuszach osiągnął dodatni bilans zwycięstw nad standardowym Negamaxem (56 do 44 dla Depth 4 oraz 52 do 48 dla Depth 5). Zdolność do przewidywania i matematycznego kalkulowania ryzyka błędu pozwala agentowi unikać ryzykownych ścieżek, które u standardowego Negamaxa prowadzą do przegranej w przypadku wystąpienia 10% szansy na potknięcie. Przewaga nie jest miażdżąca, co jest wprost proporcjonalne do relatywnie niskiego prawdopodobieństwa wystąpienia błędu (zaledwie 10%), jednak w skali makro (100 gier) daje zauważalny zysk statystyczny.

- **Złożoność obliczeniowa i narzut czasowy:** Uzyskanie przewagi analitycznej wiąże się z ogromnym kosztem wydajnościowym. Jak wynika z pomiarów, Expectiminimax wymaga nieporównywalnie więcej czasu na podjęcie decyzji. Dla głębokości równej 4, czas obliczeń wzrósł ponad dwukrotnie (0.599 s vs 0.237 s). Różnica ta staje się jeszcze bardziej drastyczna przy głębokości 5, gdzie Expectiminimax był prawie czterokrotnie wolniejszy (2.368 s vs 0.661 s). Wynika to z faktu, że dla każdego ruchu, w którym pobierano więcej niż jeden element, algorytm musiał symulować i ewaluować odgałęzienia dla dwóch różnych stanów gry (normalnego i pechowego), co skutecznie niwelowało część korzyści płynących z odcięcia Alfa-Beta i drastycznie zwiększało współczynnik rozgałęzienia drzewa gry.

Podsumowując, wybór między klasycznym algorytmem minimaxowym z odcięciem Alfa-Beta a wariantem Expectiminimax stanowi klasyczny w inżynierii oprogramowania kompromis między optymalnością decyzji w środowisku stochastycznym a złożonością czasową obliczeń.